

Consider the following two transactions:

T1: **read (A)**
 read (B)
 B := B + 0.1*A
 write (B)

T2: **read (B)**
 read (A)
 A := A – 0.05*B
 write (A)

- 1. Add lock and unlock instructions to both transactions so that they follow the two-phase locking protocol.
- 2. Can the execution of these two transactions result in a deadlock?

1

T1:	lock-S(A) read(A) lock-X(B) read(B) B := B + 0.1*A write(B) unlock(A) unlock(B)	T2:	lock-S(B) read(B) lock-X(A) read(A) A := A - 0.05*B write(A) unlock(B) unlock(A)
-----	--	-----	---

2

Eksekusi kedua transaksi dapat menyebabkan deadlock. Contoh urutan eksekusinya adalah:

Langkah	T1	T2
1	lock-S(A)	
2	read(A)	
3		lock-S(B)
4		read(B)
5	lock-X(B)	
6		lock-X(A)

- T1 memegang S-lock pada A dan meminta X-lock pada B. Namun T2 memegang S-lock pada B, dan S tidak kompatibel dengan X, sehingga T1 menunggu T2.
- T2 memegang S-lock pada B dan meminta X-lock pada A. Namun T1 memegang S-lock pada A, sehingga T2 menunggu T1.

Is the following schedule a two-phase locking (2PL) schedule (legal under 2PL protocol)? Explain.

R1(A); R2(A); R3(B); W1(A); R2(C); R2(B); W2(B); W1(C);

Ya, schedule tersebut tergolong 2PL schedule (legal di bawah protokol 2PL). Penempatan lock yang memenuhi 2PL:

1. R1(A): T1 lock-S(A)
2. R2(A): T2 lock-S(A)
3. R3(B): T3 lock-S(B), lalu unlock(B)
4. Sebelum W1(A): T2 lock-X(B), lock-S(C), lalu unlock(A)
5. W1(A): T1 upgrade lock A dari S ke X
6. R2(C), R2(B), W2(B): T2 sudah memegang lock yang diperlukan, lalu unlock(B) dan unlock(C)
7. W1(C): T1 lock-X(C), lalu unlock(A) dan unlock(C)

Setiap transaksi mengambil semua lock sebelum unlock pertamanya (tidak ada lock setelah unlock), dan tidak ada permintaan lock yang berbenturan dengan lock transaksi lain, sehingga ketiga transaksi memenuhi 2PL dan urutan jadwal tetap sama seperti soal.

Precedence graph: $T3 \rightarrow T2 \rightarrow T1$, sehingga jadwal conflict serializable, sesuai urutan lock point (T3, T2, T1).

Instructions from T1, T2, and T3 arrive in the following order.

R1(A); R2(A); R3(B); W1(A); R2(C); R2(B); C3; W2(B); C2; W1(C); C1;

What is the final schedule if the 2-phase locking with automatic acquisition of locks is implemented by CC Manager?

Asumsi:

- CC Manager memberikan lock secara otomatis sebelum operasi read/write.
- Semua lock dilepas saat commit/abort.
- Operasi yang tidak bisa mendapat lock harus menunggu, dan operasi berikutnya dari transaksi yang sama ikut tertahan.

No	Instruksi	Aksi CC Manager	Status
1	R1(A)	T1 diberi lock-S(A)	dieksekusi
2	R2(A)	T2 diberi lock-S(A)(S kompatibel dengan S)	dieksekusi
3	R3(B)	T3 diberi lock-S(B)	dieksekusi
4	W1(A)	T1 butuh upgrade ke lock-X(A), tetapi T2 masih memegang S(A)	T1 menunggu T2
5	R2(C)	T2 diberi lock-S(C)	dieksekusi
6	R2(B)	T2 diberi lock-S(B), kompatibel dengan S(B) milik T3	dieksekusi
7	C3	T3 commit, lock S(B) dilepas	dieksekusi
8	W2(B)	T2 di-upgrade ke lock-X(B), karena tidak ada transaksi lain yang memegang B	dieksekusi
9	C2	T2 commit, lock A, B, C dilepas	dieksekusi
-	W1(A) (tertunda)	T1 diberi upgrade lock-X(A)	dieksekusi
10	W1(C)	T1 diberi lock-X(C)	dieksekusi
11	C1	T1 commit, semua lock dilepas	dieksekusi

Tidak terjadi deadlock, karena hanya T1 yang menunggu ($T1 \rightarrow T2$) dan tidak ada siklus pada wait-for graph.

Jadwal akhir: R1(A); R2(A); R3(B); R2(C); R2(B); C3; W2(B); C2; W1(A); W1(C); C1;

W1(A) bergeser ke setelah C2 karena harus menunggu T2 melepas lock S(A). W1(C) dan C1 ikut mengikuti setelahnya.

Instructions from T1, T2, and T3 arrive in the following order.

R1(A); R2(A); R3(B); W1(A); R2(C); R2(B); C3; W2(B); C2; W1(C); C1;

What is the final schedule if the 2-phase locking with automatic acquisition of locks with

1. wait-die deadlock prevention scheme
2. wound-wait deadlock prevention scheme

is implemented by CC Manager?

Assume that Timestamp (T1,T2,T3)=(1,2,3)

1	dengan skema wait-die:		
	No	Instruksi	Aksi CC Manager
	1	R1(A)	T1 mendapat S(A)
	2	R2(A)	T2 mendapat S(A)
	3	R3(B)	T3 mendapat S(B)
	4	W1(A)	T1 butuh X(A), T2 memegang S(A). T1(1) lebih tua dari T2 (2), maka T1 menunggu
	5	R2(C)	T2 mendapat S(C)
	6	R2(B)	T2 mendapat S(B)(kompatibel dengan S milik T3)
	7	C3	T3 commit, lock dilepas
	8	W2(B)	T2 upgrade ke X(B)(tidak ada pemegang lain)
	9	C2	T2 commit, lock dilepas. T1 mendapat X(A), lalu W1(A) dieksekusi
	10	W1(C)	T1 mendapat X(C)
	11	C1	T1 commit
R1(A); R2(A); R3(B); R2(C); R2(B); C3; W2(B); C2; W1(A); W1(C); C1;			

2

dengan skema wound-wait:

No	Instruksi	Aksi CC Manager
1	R1(A)	T1 mendapat S(A)
2	R2(A)	T2 mendapat S(A)
3	R3(B)	T3 mendapat S(B)
4	W1(A)	T1 butuh X(A), T2 memegang S(A). T1 lebih tua, maka T2 di-wound (abort A2), lock T2 dilepas, T1 mendapat X(A), lalu W1(A) dieksekusi
5	R2(C), R2(B)	T2 di-restart dan mengulang dari R2(A). A di-lock X oleh T1 (lebih tua), sehingga T2 menunggu. Instruksi T2 berikutnya tertahan
6	C3	T3 commit, lock S(B) dilepas
7	W2(B), C2	Tertahan (T2 masih menunggu)
8	W1(C)	T1 mendapat X(C) (T2 tidak memegang lock apa pun)
9	C1	T1 commit, lock A dan C dilepas
10	T2 dilanjutkan	T2 mengulang R2(A), R2(C), R2(B), W2(B) (S(A), S(C), S(B), upgrade X(B)), lalu C2

R1(A); R2(A); R3(B); A2; W1(A); C3; W1(C); C1; R2(A); R2(C); R2(B); W2(B); C2;

Berikut ini adalah urutan kedatangan operasi-operasi dari 3 buah transaksi yang berjalan secara konkuren ke *Transaction Manager*. Ry(A) dan Wy(A) menyatakan operasi yang dilakukan oleh transaksi Ty terhadap item data A, Cy menyatakan commit transaksi Ty.

R1(X); W2(X); W2(Y); W3(Y); W1(X); C1; C2; C3;

Apabila protokol konkurensi yang digunakan adalah *two phase locking*, **tuliskan *schedule*** yang dihasilkan dari eksekusi urutan operasi di atas. *Schedule* harus memuat urutan pemberian *lock* (termasuk jenis *lock* yang diperoleh – SL atau XL), eksekusi operasi (R atau W atau C), dan pembebasan *lock* (UL). **Beri penjelasan** untuk jawaban Anda.

Asumsikan bahwa *lock* diminta saat pertama kali transaksi akan mengakses sebuah item data, dengan jenis *lock* yang disesuaikan berdasarkan keperluan akses item tersebut oleh transaksi (*exclusive lock* jika nilainya akan diubah oleh transaksi atau *shared lock* jika tidak akan diubah).

Secara umum *transaction manager* akan memproses operasi berdasarkan urutan kedatangan. Namun, jika sebuah transaksi sedang terblok (karena menunggu *lock*), semua operasi dari transaksi tersebut yang diterima oleh *transaction manager* akan diantrikan hingga transaksi bisa jalan kembali. *Transaction manager* akan melanjutkan eksekusi operasi berikutnya yang diterima. Pada saat sebuah transaksi yang tadinya terblok dapat berjalan kembali, semua operasi transaksi tersebut yang berada di antrian akan mendapat prioritas untuk dieksekusi sebelum *transaction manager* kembali melayani operasi berikutnya yang diterima.

T1	T2	T3	CC Manager
R1(X)			XL1(X)
	W2(X)		Waiting: L(X) queue: W2(X)
	W2(Y)		queue: W2(X), W2(Y)
		W3(Y)	XL3(Y)
W1(X)			W1(X)
C1			UL1(X)
	W2(X)		XL2(X)
	W2(Y)		waiting: L(Y) queue: W2(Y)
	C2		queue: W2(Y), C2
		C3	UL3(Y)
	W2(y)		XL2(Y)
	C2		UL2(X), UL2(Y)

Schedule:

XL1(X), R1(X), XL3(Y), W3(Y), W1(X), UL1(X), C1, XL2(X), W2(X), UL3(Y), C3, XL2(Y), W2(Y), UL2(Y), C2

T1 membaca data X (R1(X)) dan memperoleh shared lock (SL) pada X. Kemudian, T1 meng-upgrade lock tersebut ke exclusive lock (XL) untuk menulis ke X (W1(X)), dan setelah selesai, T1 melakukan commit dan melepaskan lock (UL1(X)). T2 berusaha menulis ke X (W2(X)) dan Y (W2(Y)). Untuk X, T2 harus menunggu hingga T1 melepaskan lock-nya karena konflik lock. Untuk Y, T2 langsung mendapatkan XL karena belum ada lock lain dan berhasil menulis. Setelah T1 melepaskan lock pada X, T2 melanjutkan dengan operasi penulisannya

ke X. T2 menyelesaikan operasi-operasinya dan melakukan commit, melepaskan semua lock yang dipegangnya pada X dan Y (UL2(X), UL2(Y)). T3 ingin menulis ke Y (W3(Y)), tetapi terhalang karena T2 telah memegang XL pada Y. T3 harus menunggu hingga T2 melepaskan lock pada Y. Setelah lock dilepaskan oleh T2, T3 memperoleh XL, menulis ke Y, melakukan commit, dan melepaskan lock (UL3(Y)).

Diberikan 2 (dua) buah transaksi berikut ini.

T1: SL(E); R(E); XL(F); R(F); XL(G); R(G); W(F); W(G); UL(F); UL(G);

T2: SL(G); R(G); SL(F); R(F); XL(E); R(E); W(E); UL(G); UL(F); UL(E);

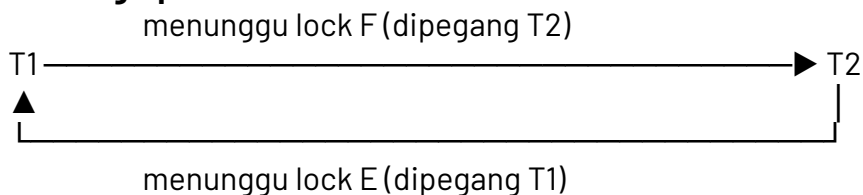
1. Perhatikan **sebuah contoh schedule konkuren** dari eksekusi kedua transaksi tersebut dengan menggunakan protokol *two phase locking* yang memiliki kondisi *deadlock*. **Perlihatkan wait-for graph** pada saat terjadi *deadlock*.
2. **Jelaskan proses deadlock recovery** yang dapat dilakukan pada kondisi *deadlock* pada poin a.
3. Dengan menggunakan *schedule* yang Anda hasilkan pada poin a, **jelaskan 2 (dua) strategi deadlock prevention** yang dapat dilakukan sehingga dapat mencegah terjadinya *deadlock*.

1

T1 tidak memiliki instruksi UL(E), sehingga diasumsikan lock E dilepas saat T1 selesai. Kedua transaksi tetap memenuhi 2PL, karena semua lock diambil sebelum unlock pertama.

No	Transaksi	Instruksi	Keterangan
1	T1	SL(E)	granted
2	T1	R(E)	
3	T2	SL(G)	granted
4	T2	R(G)	
5	T2	SL(F)	granted
6	T2	R(F)	
7	T1	XL(F)	menunggu, karena F sedang di-lock S oleh T2
8	T2	XL(E)	menunggu, karena E sedang di-lock S oleh T1

wait-for graph:



2	1. CC Manager memeriksa wait-for graph secara periodik. Ditemukan siklus $T1 \rightarrow T2 \rightarrow T1$. 2. Pilih transaksi dengan biaya rollback minimum. T1 baru melakukan 1 lock dan 1 operasi (SL(E), R(E)), sedangkan T2 sudah memegang 3 lock dan melakukan 2 operasi baca. Jadi T1 dipilih sebagai victim. Agar tidak terjadi starvation, transaksi yang paling tua dalam siklus sebaiknya tidak selalu dipilih sebagai victim. 3. Dipakai <i>total rollback</i>: T1 di-abort dan semua lock-nya dilepas (SL(E)). Partial rollback pada T1 hasilnya sama, karena lock E adalah lock pertamanya. 4. Setelah S(E) dilepas, T2 mendapat XL(E) lalu menjalankan R(E), W(E), UL(G), UL(F), UL(E). Setelah itu T1 di-restart dari awal dan dapat berjalan tanpa konflik.
3	Terdapat dua strategi deadlock prevention: a. Pre-declaration (conservative 2PL) Setiap transaksi harus mengambil semua lock yang dibutuhkan sebelum mulai eksekusi. Jika ada satu lock yang tidak tersedia, transaksi tidak memegang lock apa pun dan menunggu. • T1 meminta SL(E), XL(F), XL(G) sekaligus. Karena semuanya tersedia, T1 mendapatkannya. • T2 meminta SL(G), SL(F), XL(E) sekaligus. Karena G, F, dan E sedang dipegang T1, T2 menunggu tanpa memegang lock apa pun. • T1 selesai dan melepas lock, lalu T2 mendapat semua lock-nya dan berjalan. Tidak ada transaksi yang memegang lock sambil menunggu lock lain (tidak ada hold-and-wait), sehingga siklus tidak mungkin terbentuk. Kekurangannya, konkurensi rendah dan transaksi harus tahu semua data item di awal. b. Skema wait-die / wound-wait (berbasis timestamp) Asumsi $TS(T1) = 1$ (lebih tua) dan $TS(T2) = 2$ (lebih muda). Schedule pada poin 1 diulang sampai langkah 7: Wait-die (yang muda tidak boleh menunggu yang tua): • Langkah 7: T1 (tua) meminta F milik T2 (muda), maka T1 menunggu. • Langkah 8: T2 (muda) meminta E milik T1 (tua), maka T2 die (rollback) dan melepas lock G dan F. • T1 mendapat XL(F), lalu XL(G), dan menyelesaikan transaksinya. T2 di-restart dengan timestamp semula dan berjalan setelah T1 selesai. Wound-wait (yang tua memaksa rollback yang muda): • Langkah 7: T1 (tua) meminta F milik T2 (muda), maka T2 di-wound (rollback) dan lock G dan F-nya dilepas. T1 mendapat XL(F).

- | | |
|--|---|
| | <ul style="list-style-type: none">• T2 di-restart. Jika T2 meminta lock yang dipegang T1 (yang lebih tua), T2 hanya menunggu.• T1 menyelesaikan transaksinya, lalu T2 melanjutkan. |
|--|---|